

Projeto: WhatsApp AI — Assistente Ortopédico (Clínica CTO)

Objetivo

Construir um backend Node.js que recebe mensagens de pacientes via WhatsApp (integrado ao n8n), processa com a API do Claude e devolve uma resposta clínica contextualizada ao n8n para envio.

Stack

- **Runtime:** Node.js 20+ com TypeScript
 - **Framework:** Express
 - **IA:** Anthropic SDK ([@anthropic-ai/sdk](#))
 - **Memória de conversa:** Redis (com fallback em Map em memória se Redis não disponível)
 - **Persistência de logs:** SQLite via [better-sqlite3](#)
 - **Validação:** [zod](#)
 - **Utilitários:** [dotenv](#), [winston](#) (logs)
-

Estrutura de arquivos a criar

whatsapp-ai/

```
├── src/
|   ├── index.ts      # Entry point — inicializa Express
|   ├── routes/
|   |   ├── webhook.ts  # POST /webhook (recebe do n8n)
|   |   ├── health.ts   # GET /health (status)
|   └── services/
```

```
| | └─ claude.ts      # Chama a API do Claude com histórico
| | └─ memory.ts     # Gerencia histórico por número de telefone
| | └─ logger.ts     # Winston logger
| └─ db/
| | └─ sqlite.ts     # Cria tabela de logs e salva conversas
| └─ prompts/
| | └─ system.ts     # System prompt da clínica
| └─ types.ts        # Tipos compartilhados
└─ .env.example
└─ package.json
└─ tsconfig.json
└─ README.md
```

Comportamento do sistema

Rota principal: **POST** [/webhook](#)

Payload que o n8n envia:

```
{
  "phone": "5583999999999",
  "name": "João Silva",
  "message": "Estou com dor no joelho há 3 dias"
}
```

Resposta que o backend retorna (para o n8n enviar ao WhatsApp):

```
{  
  
  "reply": "Olá João! Entendo que você está com dor no joelho há 3 dias..."  
  
}
```

Fluxo interno:

1. Recebe e valida payload com Zod
2. Busca histórico de conversas do número no Redis (ou Map em memória)
3. Monta array de mensagens [{ **role**, **content** }] com histórico + mensagem atual
4. Chama Claude API com system prompt + histórico
5. Salva a troca (user + assistant) no histórico e no SQLite
6. Retorna { **reply**: **string** }

Limitações de memória:

- Manter no máximo **20 turnos** por número (40 mensagens: 20 user + 20 assistant)
- Se ultrapassar, descartar as mais antigas (sliding window)
- TTL do histórico no Redis: **48 horas** de inatividade

System Prompt (arquivo **src/prompts/system.ts**)

```
export const SYSTEM_PROMPT = `
```

Você é a assistente virtual da Clínica CTO – Centro de Trauma e Ortopedia,

em Campina Grande-PB. O médico responsável é o Dr. Valth Menezes Guimarães,

ortopedista e traumatologista (CRM-PB 6326, RQE-PB 5708).

SUA FUNÇÃO:

- Acolher pacientes com cordialidade e linguagem simples
- Responder dúvidas gerais sobre ortopedia, traumatologia, pós-operatório
e reabilitação de forma educativa e segura

- Ajudar a identificar se o caso é urgente (encaminhar ao PS) ou eletivo

(agendar consulta)

- Informar sobre os procedimentos realizados na clínica:

artroscopia, reconstrução do LCA, cirurgia de joelho, tornozelo, ombro,

quadril, coluna, fraturas, e bloqueios anestésicos/infiltrações guiados

por ultrassom

- Orientar sobre cuidados pós-cirúrgicos gerais

- Auxiliar no agendamento informando que o paciente deve ligar para

[INSERIR TELEFONE DA CLÍNICA] ou responder com "AGENDAR"

REGRAS CRÍTICAS:

1. NUNCA forneça diagnóstico definitivo — use sempre linguagem como

"pode ser indicativo de", "é necessário avaliação presencial"

2. Em casos de trauma agudo (fratura suspeita, luxação, lesão grave),

instrua IMEDIATAMENTE a ir ao pronto-socorro mais próximo

3. Não prescreva medicamentos — nem dose, nem nome comercial

4. Se o paciente descrever dor intensa, febre alta com dor articular,

formigamento em membro, perda de força súbita — classifique como URGENTE

5. Mantenha respostas concisas (máximo 5 parágrafos curtos)

6. Use linguagem acolhedora, sem jargões excessivos

7. Se não souber responder com segurança, diga que vai verificar com o

Dr. Valth e oriente a aguardar ou ligar para a clínica

INFORMAÇÕES DA CLÍNICA:

- Nome: Clínica CTO – Centro de Trauma e Ortopedia
- Médico: Dr. Valth Menezes Guimarães
- Especialidade: Ortopedia e Traumatologia
- Local: Campina Grande, Paraíba
- Equipamentos: Ultrassom GE Healthcare Venue 40 para procedimentos guiados

TRIAGEM RÁPIDA:

- Resposta "AGENDAR": informe como agendar consulta
- Resposta "URGÊNCIA": oriente PS imediatamente
- Resposta "EXAME": oriente que exames dependem de avaliação presencial

`trim();

Arquivo `.env.example`

ANTHROPIC_API_KEY=sk-ant-...

PORT=3000

REDIS_URL=redis://localhost:6379

CLAUDE_MODEL=claude-sonnet-4-20250514

MAX_HISTORY_TURNS=20

LOG_LEVEL=info

CLINIC_PHONE=(83) 9999-9999

Serviço de memória (`src/services/memory.ts`)

- Tentar conectar ao Redis na inicialização
 - Se Redis indisponível, usar `Map<string, Message[]>` em memória com aviso no log
 - Exportar funções: `getHistory(phone)`, `appendToHistory(phone, userMsg, assistantMsg)`, `clearHistory(phone)`
 - Aplicar sliding window: manter apenas os últimos `MAX_HISTORY_TURNS` turnos
-

Serviço Claude (`src/services/claude.ts`)

- Usar `@anthropic-ai/sdk` com `client.messages.create()`
 - Model: `process.env.CLAUDE_MODEL` (padrão: `claude-sonnet-4-20250514`)
 - `max_tokens`: 1024
 - Passar `system: SYSTEM_PROMPT` e `messages: [histórico + mensagem atual]`
 - Extrair o texto da resposta de `response.content[0].text`
 - Em caso de erro da API, retornar mensagem padrão: "No momento não consigo processar sua mensagem. Por favor, ligue para a clínica."
 - Logar tokens usados (input/output) no logger
-

Banco SQLite (`src/db/sqlite.ts`)

Criar tabela na inicialização:

```
CREATE TABLE IF NOT EXISTS conversations (
```

```
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  phone TEXT NOT NULL,
```

```
  patient_name TEXT,
```

```
  user_message TEXT NOT NULL,
```

```
  assistant_reply TEXT NOT NULL,
```

```
  tokens_input INTEGER,
```

```
tokens_output INTEGER,  
  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
  
);
```

Exportar função `logConversation(data)` que insere um registro.

Rota `/health`

Retornar:

```
{  
  
  "status": "ok",  
  
  "timestamp": "...",  
  
  "redis": "connected" | "in-memory",  
  
  "model": "claude-sonnet-4-20250514"  
}
```

README.md

O README deve conter:

1. Pré-requisitos

- Node.js 20+
- Redis (opcional mas recomendado)
- Conta Anthropic com API key

2. Instalação

```
npm install
```

```
cp .env.example .env
```

editar .env com sua ANTHROPIC_API_KEY

npm run dev

3. Como conectar ao n8n

Fluxo n8n (passo a passo):

Node 1 — WhatsApp Trigger (ou Webhook trigger)

- Configurar para receber mensagens do WhatsApp Business / Evolution API

Node 2 — HTTP Request

- Method: **POST**
- URL: **http://SEU_SERVIDOR:3000/webhook**
- Body (JSON):

```
{  
  
  "phone": "{{ $json.from }}",  
  
  "name": "{{ $json.pushName }}",  
  
  "message": "{{ $json.body }}"  
  
}
```

Node 3 — WhatsApp Send (ou Evolution API Send)

- Mensagem: **{{ \$json.reply }}**
- Para: **{{ \$node["Node 1"].json.from }}**

Variáveis de ambiente do n8n:

- A URL do webhook depende de onde o backend está rodado (local com ngrok, VPS, etc.)

4. Deploy em VPS (Ubuntu)

npm run build

pm2 start dist/index.js --name whatsapp-ai

pm2 save

5. Segurança

- Adicionar ao `.env` um `WEBHOOK_SECRET` e validar header `x-webhook-secret` nas requisições do n8n
 - Implementar rate limiting por número de telefone: máximo 10 mensagens/minuto
-

Extras a implementar (bonus, se viável):

1. **Rate limiting por telefone:** usar `Map<string, number[]>` com timestamps para bloquear spam
 2. **Detecção de palavras-chave:** se mensagem contiver "urgência", "emergência", "fratura", "sangramento" → responder com alerta de PS mesmo antes de chamar Claude
 3. **Whitelist de números:** variável de ambiente `ALLOWED_PHONES` (lista separada por vírgula) para restringir acesso durante testes
 4. **Endpoint `POST /clear/:phone`:** para limpar histórico de um número (uso administrativo)
-

Notas finais para o Claude Code

- Usar TypeScript strict mode
- Tratar todos os erros com try/catch e logar com Winston
- Não usar `any` — tipar tudo com `types.ts`
- O projeto deve rodar com `npm run dev` (ts-node ou tsx) sem necessidade de build
- Adicionar script `npm run build` para compilar para `dist/`
- Testar a rota `/health` ao final para confirmar que está funcional